Defense Record

Design Topic	1: Digital Thermometer with LED Indicators 3: Alarm System Based on Temperature Thresholds 6: Digital Clock with Temperature Display	Course Name	Curriculum Design for Embedded System & Interface Technology
		Semester	2025-2026-1
Student Name	KHAN MD SHAKIL	Student ID	2330130220

1: Digital Thermometer with LED Indicators

Question 1： How do you configure the ADC (Analog-to-Digital Converter) in the STM32L431 to read the temperature sensor's data accurately?

Answer： The ADC in the STM32L431 is important for the purpose of changing the temperature sensor's analog signal to a digital signal. The following configurations were implemented to ensure accurate data collection:

- **ADC Initialization**: The ADC is configured to run in the relative mode and the conversion of temperature sensor data will be continuous. This mode allows temperature changes to be captured live without having to activate the system manually.
- **Resolution Configuration**: The ADC has been set at 12 bits to offer a high level of information with regard to the conversion of signals. This resolution makes it possible to identify small temperature differences within the system.
- **Calibration**: According to the ADC reference voltage and the characteristics of the temperature sensor described in its datasheet, calibration is carried out. An algebraic equation converts the raw data from ADC to temperature readings.
- **Sampling Rate**: The system is configured in such a way that it always reads data from the sensor and in order to remove the noise from the raw data, averaging mechanism has been provided. The system reduces transient errors and guarantees steady values by integrating several samples.
- **Data Validation**: A scenario that is presented for controlling the readings within the optimum temperature range is the handling of the edge case so as to exclude ADC readings that may result from noise or hardware failure.

Question 2：What GPIO configuration is used for controlling the LEDs?

Answer: The GPIO configuration is very significant for the visual representation of temperature range. The following steps were taken to configure GPIO pins for LED control:

- **GPIO Initialization**: Some GPIO pins are set and configured to act as output pins in the code as shown below. These pins control three LEDs employed in the circuit of

this project. The mapping is as follows:
    - Green LED for normal temperature, assigned to `LIGHT_GREEN`.
    - Yellow LED for warm temperature, assigned to `LIGHT_GREEN & LIGHT_RED`.
    - Red LED for high temperature, assigned to `LIGHT_RED`.
- **Pin Mode Setting**: All the GPIO pins are configured to output mode which allows the microcontroller to either provide current to light the LEDs or draw current from the LEDs. It is done through the programming of the output logic levels to either on or off the LEDs.
- **LED Control Logic**: With the given temperature ranges for example 0°C to 20°C for normal, the GPIO pin associated with this range is set to high while other are set to low. This way, only one LED is on at any one time.
- **Debounce Logic**: In this code, a strategy of delaying has been added between the temperature ranges to prevent flickering, which results in clear and understandable LED behavior.

Question 3：How does the STM32L431's low-power mode affect the performance of the thermometer and LEDs?

Answer: The low power mode of the STM32L431 is intended for energy-saving, which is important for the devices operated from the battery or with the limited consumption of energy. However, it introduces certain trade-offs that can affect the performance of the thermometer and LEDs:

- **Impact on ADC**: During the low power mode, the ADC may work at a lower sampling rate, and/or with a fewer number of bits, which may limit the system's ability to respond quickly and with high accuracy to temperature changes.
- **Impact on LEDs**: The GPIO pins which are used to control the LED may also be slow in response or may take some time to respond due to the low clock speeds in the low-power mode. This could lead to some minor latency of LED state changes but does not limit the entire functionality.
- **Mitigation Strategies**: The system design allows the ADC and GPIO for instance to be operational even during low power mode. Furthermore, the wake-up mechanisms are incorporated in order to provide complete functionality if required.
- **Power Efficiency**: Although some performance degradation is observed, the low-power mode significantly improves the energy efficiency of the system and is therefore appropriate for applications where extended run-time is important.

Question 4：What strategies were implemented to debounce or filter noisy signals from the temperature sensor?

Answer: Noise in the sensor data can cause a wrong temperature reading and unpredictable LED patterns. The following strategies were implemented to address these challenges:

- **Signal Averaging**: The same value of ADC is obtained several times with a small-time interval and then the average value is calculated to minimize the effect of spike noise. Averaging allows erasing of peaks and troughs and gives a constant value for temperature calculation.
- **Software Filtering**: Simple software filters, which include adding a small delay between two consecutive ADC readings, is used to give the sensor output time to settle. This makes it easier to remove noise in the system due to fast signal oscillations.
- **Calibration Adjustments**: The calibration process is further refined by using reference temperature points and sensor specific characteristics obtained from the datasheet. This helps to avoid the noise to affect ADC values to temperature outputs in the system.
- **Debounce Logic for LEDs**: To prevent flickering LEDs due to fluctuations in temperature, the system employs a delayed approach to retain the LED states for adequate time. This is done to avoid many LED changes due to small temperature fluctuations.
- **Error Detection**: Some extra measures are included to detect and handle erroneous sensor data as well. For example, if the ADC is generating values that are out of the normal range, then the system does not display the temperature which is incorrect.

3: Alarm System Based on Temperature Thresholds

Question 1： How did you set up the STM32L431's interrupt system to detect temperature threshold violations in real-time?

Answer: To ensure real-time detection of temperature threshold violations, the interrupt system of the STM32L431 was configured as follows:

- **ADC Interrupts**: The ADC was set in the free running mode to read the temperature sensor without the need of a trigger. Interrupts were employed to make events happen at specific time when the ADC conversion was complete. This reduces the time delay in identifying temperatures variations in the ambient condition.
- **Threshold Comparison in ISR**: Specifically, in the ISR, the ADC value was directly compared to the temperature threshold set prior. In the present scenario, when the threshold was crossed the system raised an alarm by setting the GPIO pin associated with the red colored LED.
- **Low-Latency Processing**: Through the use of interrupts the system was able to guarantee that the temperature threshold violation was detected and acted upon without having to wait for polling. This is because, such applications like alarms are top priority and should be handled with haste.
- **Interrupt Priorities**: It was assigned with highest priority to make sure that temperature measurement would have higher priority than for example UART debugging or non-critical GPIO operations.

Question 2：How do you configure the STM32L431 to trigger an external alarm (e.g., buzzer or LED) upon threshold breaches?

Answer: Triggering an external alarm involves the following steps:

- **GPIO Configuration**: The GPIO pin that connects with the red LED was set to output. This makes it possible for the microcontroller to change the state of the LED in relation to the temperature.
- **Threshold Logic Implementation**:
  - The main loop or ISR is always checking the converted temperature values.
  - If the temperature is above the provided threshold (in this case 20 C), the GPIO for the red LED is set high in order to switch on the alarm.
  - When the temperature has stabilized, the GPIO pin is set to 0 which switches off the LED.
- **External Alarm Integration**: In this project, a red LED is used as the alarm and the same pin could be connected to other alarms like a buzzer. This would only need some circuit changes and of course no big alteration in the codes.
- **Real-Time Feedback**: The alarm status is also logged and the information is transmitted via UART to give the user information on the status of the system as it operates.

Question 3： What role does the STM32L431's low-power mode play in monitoring temperature continuously?

Answer: The low power mode is paramount in improving the energy efficiency of the system especially in battery dependent or in low power consumption scenarios. The role of low-power mode in this system is as follows:

- **Continuous Monitoring with Reduced Power Consumption**:
  - The STM32L431 has low power modes namely the Stop and Standby modes. These modes cut the system clock frequency or turn off some unimportant hardware blocks while preserving the ADC functionality.
  - The ADC can be configured to work with low power while still providing real time temperature monitoring.
- **Wake-Up Mechanism**: Some of the interrupts such as ADC or GPIO can wake the microcontroller from low power mode when there is a large change in temperature. It maintains high-power response levels without having to operate at high power continuously.
- **Balancing Performance and Power**: During the low-power mode, the clock frequency of non-critical peripheral blocks (e.g., UART) is also decreased. However, focusing on the ADC and GPIO, the system delivers precise data acquisition with alarm generation.
- **Long-Term Operation**: Through efficient use of low power modes, it is possible to run the system for long durations without the need for frequent battery replacement

or charging which is suitable for self-contained and remote-controlled applications.

Question 4：How do you implement hysteresis in the alarm system to prevent frequent toggling near threshold values?

Answer: Hysteresis was implemented to stabilize the system's behavior near the threshold values and avoid frequent toggling of the alarm:

- **Temperature Range for Hysteresis**: A small buffer range was incorporated around the threshold level. For instance, if the threshold is at 20°C then the alarm is triggered only when the temperature is above 21°C and only disengaged when the temperature is below 19°C.
- **Software Implementation**: The logic for hysteresis was hard coded in the threshold comparison code:
    - If temperature exceeds the upper limit for instance 21°C then an alarm is raised.
    - The alarm is still in effect until the temperature goes below the lower bound (e.g., 19°C).
- **Benefits of Hysteresis**:
    - It avoids frequent switching on and off of the LED alarm particularly during small and temporary changes in the temperature close to the set levels.
    - This makes the system more reliable and the user to have a better experience because there are no unmanned alarms.

Question 5：How does the system ensure accurate threshold settings, and how can they be updated dynamically?

Answer: The accuracy and flexibility of threshold settings are ensured through the following mechanisms:

- **Predefined Constants**: Threshold values are initially set as constants in the code for instance (e.g., `#define TEMP_HIGH 20`). These two are selected depending on the application of the system and the properties of the sensors to be used.
- **Calibration**: The thresholds are set with regard to real temperature conditions during testing. This brings the system closer to the expected performance and thus guarantees that the system behaves in the right way.
- **Dynamic Update Capability**:
    - This allows for changes in the threshold values since the UART interface is used. Threshold settings may be programmed and changed remotely through UIART interface and the microcontroller interprets the commands to adjust the thresholds without having to reprogram the device.
    - This feature makes the system versatile in that it can suit certain operational conditions or conditions on the environment.
- **Real-Time Feedback**: For this, the threshold values and the current temperature readings are shown on UART, so it is easy to monitor and modify the parameters if necessary.
- **Error Handling**: The system also has measures against wrong or inapplicable

threshold settings for the purpose of improved correctness. If such values are detected, then the system keeps the last good configuration which was defined.

6: Digital Clock with Temperature Display

Question 1： How do you configure the STM32L431's RTC for accurate timekeeping, and how is it calibrated?

Answer： The STM32L431's RTC is configured as follows to ensure accurate and reliable timekeeping:

- **RTC Initialization**: The integrated RTC module is enabled during the booting of the system. It is also used to generate a stable reference frequency which may be obtained from a low power crystal oscillator (LSE) or from the internal RC oscillator (LSI).
- **Time and Date Settings**: The RTC has the initial time and date set which are the basic for further time and date calculation.
- **Interrupt-Driven Updates**: A timer interrupt is used to update the time every second for instance. This interrupt causes periodic updates to occur, so that time is always in check.
- **Calibration**: For the drift over a long time, the RTC is synchronized with a much more precise external clock. This entails determining the difference between the RTC's output and a standard and then adjusting the pre-scaler or the clock source in a bid to enhance the long-term stability.
- **Error Handling**: There are measures in place for the correction of errors emanating from resets or power failure in order to maintain the clocks' synchronization.

Question 2： How do you multiplex the display of time and temperature using the STM32L431's GPIOs or timers?

Answer: Multiplexing the display of time and temperature involves alternating between the two outputs using GPIOs and timers:

- **GPIO Configuration**: As for the display segments or LEDs, the GPIO pins are set as output. Two different pins are provided for the time and temperature data.
- **Timer-Based Switching**: A timer interrupt is made to switch between time and temperature display at regular interval of time say one second. This will allow both the values to be displayed without the two overlapping or one blinking on top of the other.
- **Data Format**:
  - Time is displayed in the format `HH:MM:SS`.
  - Temperature is displayed in the format `XX.X°C`.
- **Control Logic**: The time interrupt causes the reading of the current time from the RTC and the temperature from the ADC. Such values are then rendered on the

display screen one by one.

- **Optimization**: The system used efficient GPIO toggling and little delay to make sure that the time and temperature display changes are smooth.

Question 3：What considerations were made to ensure low power consumption during clock operation?

Answer: Reduced power utilization is essential for the purpose of sustaining prolonged functionality, particularly in systems that are either standalone or designed for battery backup. The following strategies were implemented:

- **Low-Power RTC Mode**: The RTC is a separate block that functions in low-power mode, synchronous with the clock signal from the main processor, but it is also capable of functioning when other parts of the system are powered off.
- **Sleep and Stop Modes**: The STM32L431 is programmed to enter low power modes when there are no activities to perform. The current mode is mainly used to support only the basic peripherals like the RTC and the ADC.
- **Peripheral Optimization**:
    - The ADC is configured to work only in the temperature sampling intervals, thus saving power consumption in the circuit.
    - The UART for debugging is used at the lower baud rates in order to reduce power consumption.
- **Efficient Clock Source**: The fact that the RTC employs the LSE or LSI guarantees low power consumption but with high precision in its timing function.

Dynamic Frequency Scaling: The microcontroller operates as a clocked device and the clock frequency is variable to enable the minimum power to be consumed for low processing loads

Question 4：How does the STM32L431 handle system resets while preserving accurate time and temperature data?

Answer: The system is designed to maintain accurate time and temperature data even after resets:

- **RTC Backup**: The STM32L431 has a RTC module which is connected with a backup power source like a coin cell battery to keep the time even if the main power is reset or is switched off. This means that the clock still runs well even when there is power failure in the main power source.
- **Non-Volatile Storage**:
    - The latest temperature data is kept in non-volatile memory such as Flash or EEPROM at a certain interval.
    - At the system restart, the current temperature data is pulled from the memory and shown until the next update from the ADC.
- **Reset Detection**: A flag is used to check if the system has gone through a reset. If a reset is detected the system does not perform any initialization that may erase the RTC data.
- **Re-Synchronization**: In a case where the system identifies a time drift issue caused

by extended power failure, it provides for manual or automatic re-timing from an external source.

- **Error Handling and Logging**: All the resets are controlled and logged via UART so that all errors can be seen and recorded so proper fixes can be implemented for the system.